

</ Data Mining: Practical Python for Beginners

Instructor:
Fateme Khodabande

/>

} /> [

fatemever2001@gmail.com

01

Matplotlib

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Introduction

- Visualization is a universal tool for investigating and communicating results of computational studies, and it is hardly an exaggeration to say that the end product of nearly all computations – be it numeric or symbolic – is a plot or a graph of some sort. It is when visualized in graphical form that knowledge and insights can be most easily gained from computational results. Visualization is therefore a tremendously important part of the workflow in all fields of computational studies.
- The most popular general-purpose visualization library is Matplotlib, which mainly focuses on generating static publication-quality 2D and 3D graphs. Many other libraries focus on niche areas of visualization. A few prominent examples are Bokeh (<http://bokeh.pydata.org>) and Plotly (<http://plot.ly>), which both primarily focus on interactivity and web connectivity, Seaborn (<http://stanford.edu/~mwaskom/software/seaborn>) which is a high-level plotting library which targets statistical data analysis and which is based on the Matplotlib library, and the Mayavi library (<http://docs.enthought.com/mayavi/mayavi>) for high-quality 3D visualization, which uses the venerable VTK software (<http://www.vtk.org>) for heavy-duty scientific visualization.

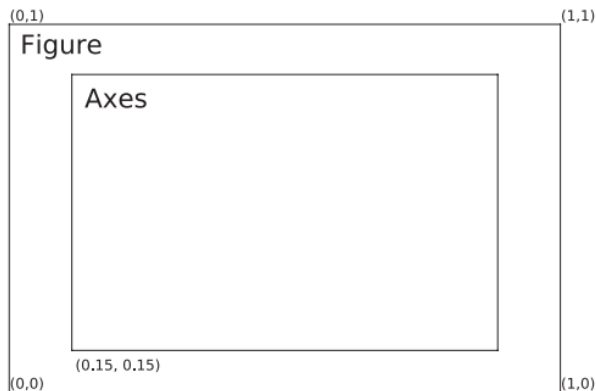
Importing Modules

Unlike most Python libraries, Matplotlib actually provides multiple entry points into the library, with different application programming interfaces (APIs). Specifically, it provides a stateful API and an object-oriented API, both provided by the module *matplotlib.pyplot*. I strongly recommend to only use the object-oriented approach, and the remainder of this file will solely focus on this part of Matplotlib

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Getting Started

A graph in Matplotlib is structured in terms of a Figure instance and one or more Axes instances within the figure. The Figure instance provides a canvas area for drawing, and the Axes instances provide coordinate systems that are assigned to fixed regions of the total figure canvas;



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Figures & Axes

- A Figure can contain multiple Axes instances, for example, to show multiple panels in a figure or to show insets within another Axes instance. An Axes instance can manually be assigned to an arbitrary region of a figure canvas, or, alternatively, Axes instances can be automatically added to a figure canvas using one of several layout managers provided by Matplotlib.
- The Axes instance provides a coordinate system that can be used to plot data in a variety of plot styles, including line graphs, scatter plots, bar plots, and many other styles. In addition, the Axes instance also determines how the coordinate axes are displayed, for example, with respect to the axis labels, ticks and tick labels, and so on. *In fact, when working with Matplotlib's object-oriented API, most functions that are needed to tune the appearance of a graph are methods of the Axes class.*

Figures

- As introduced in the previous section, the Figure object is used in Matplotlib to represent a graph. In addition to providing a canvas on which, for example, Axes instances can be placed, the Figure object also provides methods for performing actions on figures, and it has several attributes that can be used to configure the properties of a figure.
- A Figure object can be created using the function *plt.figure*, which takes several optional keyword arguments for setting figure properties. In particular, it accepts the *figsize* keyword argument, which should be assigned to a tuple on the form (width, height), specifying the width and height of the figure canvas in inches.

Axes

- The Figure object introduced in the previous section provides the backbone of a Matplotlib graph, but all the interesting content is organized within or around Axes instances.
- The Axes object is central to most plotting activities with the Matplotlib library. It provides the coordinate system in which we can plot data and mathematical functions, and in addition it contains the axis objects that determine where the axis labels and the axis ticks are placed.
- The functions for drawing different types of plots are also methods of this Axes class.
- Matplotlib provides several different Axes layout managers, which create and place Axes instances within a figure canvas following different strategies.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

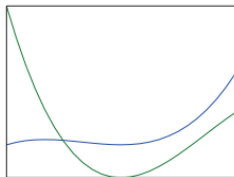
Axes

- The *plt.subplots* function is also capable of filling a figure with a grid of Axes instances, which is specified using the first and the second arguments, or alternatively with the *nrows* and *ncols* arguments, which, as the names imply, create a grid of Axes objects, with the given number of rows and columns.

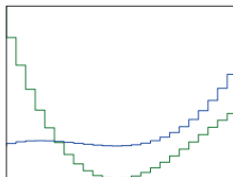
1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Axes

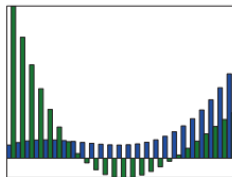
`Axes.plot`



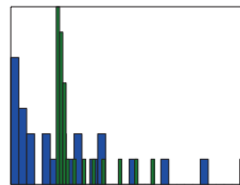
`Axes.step`



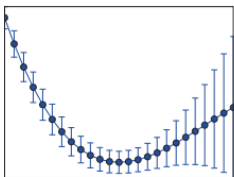
`Axes.bar`



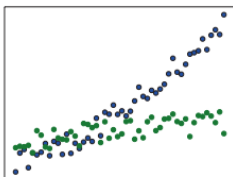
`Axes.hist`



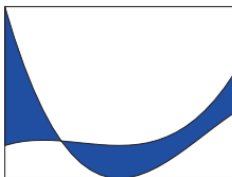
`Axes.errorbar`



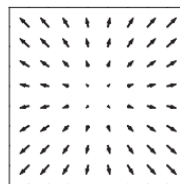
`Axes.scatter`



`Axes.fill_between`



`Axes.quiver`



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 1 1 0 1

Line Properties

- The most basic type of plot is the simple line plot.
- It may, for example, be used to depict the graph of a univariate function or to plot data as a function of a control variable. In line plots, we frequently need to configure properties of the lines in the graph, for example, the line width, line color, and line style (solid, dashed, dotted, etc.). In Matplotlib we set these properties with keyword arguments to the plot methods, such as `plot`, `step`, and `bar`.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Line Properties

Argument	Example Values	Description
color	A color specification can be a string with a color name, such as "red," "blue," etc., or a RGB color code on the form "#aabbcc."	A color specification.
alpha	Float number between 0.0 (completely transparent) and 1.0 (completely opaque).	The amount of transparency.
linewidth, lw	Float number.	The width of a line.
linestyle, ls	"-" – solid "--" – dashed "." – dotted "-." – dash-dotted	The style of the line, i.e., whether the line is to be drawn as a solid line or if it should be, for example, dotted or dashed.
marker	+, o, * = cross, circle, star s = square . = small dot 1, 2, 3, 4, ... = triangle-shaped symbols with different angles.	Each data point, whether or not it is connected with adjacent data points, can be represented with a marker symbol as specified with this argument.

Argument	Example Values	Description
markersize	Float number.	The marker size.
markerfacecolor	Color specification (see in the preceding text).	The fill color for the marker.
markeredgewidth	Float number.	The line width of the marker edge.
markeredgecolor	Color specification (see above).	The marker edge color.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Text Formatting and Annotations

- Text labels, titles, and annotations are important components in most graphs, and having full control of, for example, the font types and font sizes that are used to render such texts is a basic requirement for producing publication-quality graphs.
- It may, for example, be used to depict the graph of a univariate function or to plot data as a function of a control variable. In line plots, we frequently need to configure properties of the lines in the graph, for example, the line width, line color, and line style (solid, dashed, dotted, etc.). In Matplotlib we set these properties with keyword arguments to the plot methods, such as `plot`, `step`, and `bar`.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Text Formatting and Annotations

Argument	Description
fontsize	The size of the font, in points.
family or fontname	The font type.
backgroundcolor	Color specification for the background color of the text label.
color	Color specification for the font color.
alpha	Transparency of the font color.
rotation	Rotation angle of the text label.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Axis Properties

- After having created Figure and Axes objects, the data or functions are plotted using some of the many plot functions provided by Matplotlib, and the appearance of lines and markers are customized – the last major aspect of a graph that remains to be configured and fine-tuned is the Axis instances.
- A two-dimensional graph has two axis objects: for the horizontal x axis and the vertical y axis. Each axis can be individually configured with respect to attributes such as the axis labels, the placement of ticks and the tick labels, and the location and appearance of the axis itself.
- Axis Labels and Titles
- Axis Range
- Axis Ticks, Tick Labels, and Grids
- Twin Axes
- Insets

</ Reference: Numerical Python />

<https://doi.org/10.1007/978-1-4842-4246-9>

